

Swarm-Based Meta-Heuristics for Multi-Objective Route Optimisation in Micromobility Systems

Ben Crane
School of Computing
Dublin City University
ben.crane2@mail.dcu.ie

Hemandh Babu Babu
School of Computing
Dublin City University
hemandhbabu.babu2@mail.dcu.ie

Abstract

Micromobility modes such as e-scooters and e-bikes have reshaped urban transport by improving first- and last-mile connectivity and supporting sustainable mobility. Despite this, safety improvements haven't kept pace: this is particularly true in mixed-traffic environments where infrastructure, regulation and rider behaviour vary widely. Conventional countermeasures like speed limits and geofencing remain ineffective in dynamic settings because they rely on static, distance- or time-based optimisation. To move beyond these countermeasures, this paper evaluates whether swarm-based methods like Particle Swarm, Ant Colony and Wolf-Pack Optimisation can support adaptive, multi-objective routing for micromobility users under sparse-data and uncertain conditions. Across tests measuring computation efficiency, route length and safety improvement, Dijkstra produced the strongest results for static, pre-computed routing, while Particle Swarm Optimisation performed best under real-time conditions. These results demonstrated that swarm-based optimisation could offer a viable foundation for context-aware micromobility route optimisation.

Keywords: Dynamic Routing, Swarm Algorithms, Multi-Objective Optimisation, Micromobility Safety,

1. Introduction

Shared and privately owned micromobility systems, including e-scooters and e-bikes, have transformed urban transportation, offering first-and-last-mile connectivity, reduced congestion, lower emissions and improved health outcomes [1]. Safety, however, has not kept pace: irresponsible rider behaviour, inconsistent regulation application, inadequate infrastructure and incomplete accident data all contribute, where the ITF-OECD notes that only crashes requiring hospital treatment are consistently reported [2]. Such concerns risk hindering broader adoption [1]. Traditional countermeasures such as speed limits, parking requirements, helmet campaigns and user education have shown limited effectiveness, particularly when applied inconsistently across jurisdictions [3]. While routing tools can reduce exposure to high-risk areas without sacrificing convenience [4], conventional algorithms such as Dijkstra's and A* struggle with the dynamic, uncertain and data-limited conditions inherent to micromobility [2, 5]. This raises a

central question: how can safety be improved in unpredictable urban settings while maintaining convenience?

Although swarm intelligence is widely applied to robotic navigation, path formation and task allocation [6], its use in real-time micromobility routing remains underexplored, with related work addressing charging station planning [7] and demand management [8] rather than safety. Given its decentralised, self-organising and adaptive nature, swarm intelligence holds strong potential for addressing these limitations [6]. This paper addresses two research questions:

RQ1: What is the impact of data availability on the performance of meta-heuristic route algorithms for micromobility safety?

RQ2: To what extent can swarming algorithms reduce user risk without increasing the trip distance?

To address these questions, we processed an Irish accident dataset into safety maps, benchmarked two standard and three swarm algorithms across eight route pairs in four Irish cities and simulated adaptive changes to compare their suitability for micromobility routing.

The paper is laid out as follows: Section 2 deals with the literature review in the area of micromobility systems, the data limitations, the pre-existing measures and the algorithms in use, Section 3 documents the Methodology, Section 4 presents the results and a discussion of the main findings, Section 5 gives the Conclusion and possible Future work and Section 6 deals with the Student Contributions and Generative AI disclosure.

2. Literature review

2.1. Current State of Micromobility Systems

Recent data reinforces the above-mentioned safety challenges. The North American Bikeshare and

Scotershare Association (NABSA) recorded 225 million shared micromobility trips across 415 North American cities in 2024, two thirds of them on e-devices [9]. In Ireland, the Road Safety Authority's (RSA) 2024 national survey determined 5% of adults had used an e-scooter in the past year, predominantly in urban areas and with 75% of users aged under 35: of these, 24% reported a collision and 32% a near miss [10]. About half of respondents strongly agreed that e-scooterists exhibit risky behaviours such as distracted riding, footpath use and excessive speed, while very few agreed that riders demonstrated compliance with road safety [10]. Compounding this, cross jurisdictional accident data remains incomplete, underreported or misclassified [2, 11], with the NTSB identifying these shortcomings as a barrier to implementing effective safe-system countermeasures [11]. These safety challenges highlight the need to examine how current interventions address the risks faced by micromobility users.

2.2. Limitations of Existing Safety Measures

Current interventions such as speed limits, geofencing, parking regulations and user education provide rapid short-term responses but prove limited, blocking better solutions as regulation follows innovation rather than anticipating it [3]. Some authors proposed the use of YOLOv5 to photograph the streets and then the neural network Bikeable to assign a safety score to that street, but this is constrained by image quality issues and its failure to account for rider preferences or individual behaviour [1]. Conventional routing algorithms such as Dijkstra's and A* perform well in small, static environments but lack the adaptability required for the dynamic, uncertain conditions of real-world micromobility routing [5]. Collectively, these constraints highlight the need for adaptive optimisation approaches capable of operating effectively in modern, data-limited urban environments.

2.3. Data Limitations and Algorithms:

A persistent challenge across proposed solutions is data quality. Both the NTSB and ITF-OECD highlight misclassification, underreporting, inconsistent reporting criteria and inadequate risk exposure metrics in available micromobility data [2, 11], constraining any data-dependent intervention. Existing solutions are also insufficiently targeted. Dikshit et al.'s AI-based approach addresses dynamic route optimisation and congestion prediction but raises privacy concerns and treats traffic in isolation rather than within a micromobility-specific context [12]. Chau et al. likewise show meta-heuristics to solve multimodal routing efficiently, yet their framework

is not micromobility-specific, lacks real-time data integration and leaves scalability and deployment unresolved [13]. These gaps motivate examining swarm intelligence for micromobility route optimisation, particularly given its potential in data-limited and rapidly changing environments.

2.4. Related Works on Routing Algorithms

To address the first research question, this section reviews swarm algorithms: nature-inspired meta-heuristics in which simple, decentralised agent interactions yield complex global patterns without centralised control or scalability constraints [14]. Such properties suit micromobility route optimisation, where rider behaviour, traffic conditions, infrastructure quality and ridership vary widely and where safety-aware routing has shown potential to reduce exposure to hazardous corridors [1, 4]. As micromobility safety data is typically sparse, uneven or streamed in real time, the review attends particularly to each algorithm's dependence on available data. Three algorithms widely applied to dynamic optimisation are considered: Particle Swarm Optimisation (PSO), Ant Colony Optimisation (ACO) and Wolf-Pack Optimisation (WPO).

PSO, from Kennedy and Eberhart's early-1990s modelling of flocking behaviour, has each particle adjust its position toward its own best historical location and the best found by its neighbours [15, 16]. Operating in continuous space, it balances exploitation with exploration and is simple to implement. Because it needs only a lightweight fitness signal rather than a rich prior model, it suits conditions where accident data is limited but real-time inputs are available, a regime matched by Luo et al.'s Industrial IoT routing, which handles real-time data under multiple constraints including distance restrictions [17].

ACO, introduced by Dorigo et al. in the early 1990s, models pheromone-based foraging in ant colonies [18]. Ants select paths probabilistically from pheromone intensity and heuristic information such as distance, with trails reinforced iteratively until convergence. This accumulated pheromone functions as learned experience, partly compensating for thin prior data, but reliable trails require sufficient iterations, limiting ACO where solutions are needed quickly. Operating in discrete space, it maps naturally onto street-network routing: Liu et al.'s 3D pipe-layout application shows its capacity to escape local optima and improve convergence speed [19].

WPO, developed by Wu et al., models cooperative wolf-pack hunting through three phases: wandering, calling and besieging [20]. Wolves explore independently, then converge on a promising position, after which the

best-performing wolf leads an intensified search, with weaker wolves replaced to maintain diversity and prevent stagnation [21]. Where PSO and ACO are shaped mainly by the data present at the outset, WPO's structured exploration and replacement mechanisms target robustness when the data shifts, sustaining adaptability rather than depending on just data volume [16]. Mukha et al.'s dynamic staff-scheduling application demonstrated this, handling fluctuating demand through adaptive decision-making [22].

Applying swarm algorithms to micromobility routing is not without limitations. Shared challenges include premature convergence, computational overhead and stochastic variability [16], while WPO adds parameter sensitivity affecting runtime and scalability [21]. More fundamentally, the available case studies are predominantly non-transportation and rarely tested under the data-scarce conditions typical of micromobility safety, leaving open how far each algorithm's performance depends on data availability and quality. Establishing this dependency is a precondition for the subsequent question of whether such algorithms can improve safety without hindering convenience. These gaps motivate the methodological framework outlined in the next section.

3. Methodology

To answer the two research questions, accident data was used to build a local dataset for evaluating two standard algorithms (A* and Dijkstra) and three swarming algorithms (PSO, ACO and WPO). While Dijkstra's algorithm establishes a baseline by expanding equally in all directions to guarantee the absolute shortest path, A* optimises this search by using a heuristic to prioritise paths toward the destination. To implement these algorithms, this required dataset preprocessing and safety map construction, establishing the fastest route baseline and a means to quantify the safest.

3.1 Preparation

3.1.1 Obtaining the data

The dataset was obtained from the Active Travel Collision Tracker, a citizen-reporting platform jointly operated by the Dublin Inquirer, IrishCycle.com and the UCD Spatial Dynamics Lab [23]. It comprised 2,058 incidents reported across Ireland between 2001 and 2025 (as of September 2025), each recording the date, location (latitude/longitude), description, incident type, outcome and cause category. Citizen reporting was preferred because official active travel collision statistics are known to be under-reported [23].

3.1.2 Exploratory Data Analysis

Exploratory data analysis characterised the dataset prior to the safety map construction. Dates were standardised using the `dateutil` parser [24] and records dated before 2020 were excluded, restricting the analysis to the period following the commencement of Ireland's national active travel investment programme, under which over €1bn and more than 600 km of active travel infrastructure have been delivered since 2020 [25]. This reduced the dataset from 2,058 to 1,953 records. Missingness was limited: 311 records provided a date without a time component (no Hour, Minute or Second value) and 5 lacked MultiParty. Near-misses dominated (1,086: 56%), ahead of collisions (436) and hazards (431) and severity was strongly skewed toward minor outcomes (1,599 severity-1 cases: 82%). Driver Behaviour (Car) was the leading cause category (1,044), followed by Infrastructure (553), with monthly peaks in February (202) and March 2024 (188). These findings established the baseline for the subsequent mapping stage.

3.1.3 Algorithmic Preparation

Both conventional algorithms required a safety map as a prerequisite, with [1] mapping accident data to risk scores to identify safer routes. The map was built from Irish road network data using OSMnx, which downloads, models and visualises OpenStreetMap data as a node and edges (with intersections and dead-ends and street segments) [26]. Geocoding used Nominatim rather than Overpass, as OSMnx relies on it to resolve place names into boundaries [27]: `geocode_to_gdf` retrieved the OSM elements for Ireland, `graph_from_polygon` generated a simplified drive-network graph (chosen over the cycle network, which is disconnected), `distance.nearest_nodes` vectorised the matching of accident coordinates to nearest nodes and `graph_to_gdfs` produced an edge GeoDataFrame, since street segments rather than dead-ends were the mapping target [28]. Polars were used to process the resulting large OSM edge DataFrame alongside accident severity data [29]. Accident outcomes were converted into numeric, colour-coded severity levels and aggregated at node level, so each segment inherited the maximum severity of its connecting nodes. The map was initialised at the mean accident coordinates and each segment rendered as a colour-coded PolyLine in Folium, a Python wrapper for Leaflet.js supporting interactive HTML rendering [30, 31], with severity-zero segments omitted. The result was shown in Figure 1 and the routes for the algorithmic testing were chosen based on the map's incident clusters and the number of incidents of increasing severity (from Severity-1 to Severity-4). With the setup established, the parameters governing each algorithm were then specified.

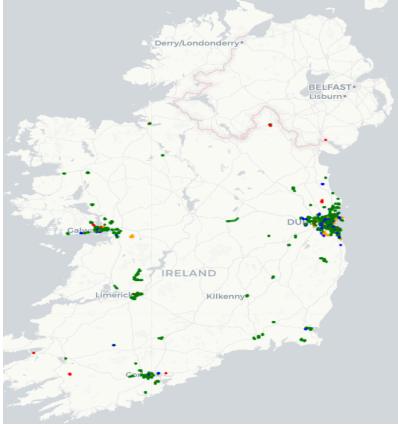


Figure 1: All-Ireland incident map that decided the testing routes

3.1.4 Parameters and Reasons

Edge severity (0–4) was mapped to an incident probability via:

$$\mathbf{p_inc} = \mathbf{clamp}(\mathbf{p_0} + \lambda \cdot \mathbf{severity}, \mathbf{0}, \mathbf{0.99}) \quad (1)$$

with $p_0 = 0.001$ and $\lambda = 0.02$

which gave severity-zero edges a small baseline while scaling risk linearly, per [1]. This became an additive cost through:

$$\mathbf{risk_weight} = -\ln(1 - \mathbf{p_inc}) \quad (2)$$

By letting standard shortest-path solvers minimise cumulative risk rather than distance [32]; the 0.99 ceiling kept this term finite on the highest-severity edges.

Dijkstra and A* needed no tuning, being deterministic and exhaustive over non-negative weights. A* was guided by an admissible great-circle heuristic (`ox.distance.great_circle`), narrowing search while preserving optimality [32]. Both formed the static-routing benchmark for evaluating the swarm methods [5].

Across the three swarm algorithms, population and iteration budgets were kept comparable (35 particles/ants, 40 wolves; up to 80 iterations) to isolate search strategy from computational allowance (RQ1, RQ2). Neighbour sampling was capped at six nodes and path length at 300–400 steps for tractability.

> PSO: inertia 0.6, equal cognitive/social coefficients of 1.4 [16]. As PSO is natively continuous [15], velocity was reinterpreted as the number of mutation operations (splice or swap) per iteration, drawing each particle toward its personal- and the global-best path.

> ACO: $\alpha = 1.0$, $\beta = 2.5$ (higher β biasing early search toward low-cost edges while pheromone is sparse), $\rho = 0.15$, $Q = 100$ [18], with early stopping after a 15-iteration stagnation limit.

> WPO: 40 wolves, top 20% retained as elites [21], with phases operationalised as random-walk construction/tail-regrowth (wandering), leader crossover

(calling/besieging) and 2-opt shortcut removal (refinement) [33] which were chosen to test resilience to local-optima stagnation relative to PSO/ACO [21].

Real-time variants used identical parameters but split each run into two cases like incidents excluding the target city and the full dataset, to isolate the effect of data availability (RQ1). WPO's fallback generator was switched from unweighted BFS to weight-aware Dijkstra for consistency with the routing objective.

3.2 Algorithmic Development (Pre and Post Real-Time)

3.2.1 Dijkstra

The algorithm paired Dijkstra with incident-severity data to produce distance- and risk-optimal routes, with the highest-incident-density city selected as an upper-bound stress test to maximise observable impact. Incident records were loaded as an EPSG:4326 GeoDataFrame and clipped to the city boundary: each edge inherited its nearest incident's severity, mapped to a probability ($p = p_0 + \lambda \cdot \text{severity}$, capped at 0.99) and converted to an additive risk weight via $-\log(1-p)$ [36, 37] where p was the probability of encountering a high-severity edge, p_0 was the edge baseline probability and λ was the scaling factor to influence severity probability, this was capped at 0.99 to avoid deterministic behaviour. Parallel edges collapsed by retaining the shortest length, lowest risk weight and highest severity. Dijkstra was then run for distance and for risk, with runtimes recorded and high-severity edge counts quantifying risk reduction. Although informally termed "probabilistic Dijkstra," the method was deterministic, using fixed weights rather than expected-cost optimisation [41] and so differed from real-time routing with continuously updated weights [42]. A two-pass design instead isolated the effect of incident data by first excluding then reintroducing it, drawing conceptual motivation from concept-drift adaptation, where outputs are re-evaluated as the data distribution changes [43], here approximated discretely. Implementation used standard geospatial and graph libraries (OSM projections [34], NetworkX [35, 38, 39], GeoPandas [44]), with visualisation in Folium [40].

3.2.2 A*

The A* implementation reused the Dijkstra pipeline in full: GeoDataFrame loading, incident-to-severity mapping, risk weighting, MultiDiGraph-to-DiGraph simplification and all downstream steps. Its one substantive addition was a heuristic: great-circle distance between coordinate pairs via `ox.distance.great_circle` [45], with `nx.astar_path` returning the shortest and safest paths [46]. Because of the heuristic estimated geographic

distance only, it remained admissible and consistent for the distance-optimal search but offered no optimality guarantee for the risk-weighted search, risk weight and physical distance being independent: it nonetheless reduced the search space relative to uninformed expansion. The same two-pass design as Dijkstra [42, 43] applied: `run_routing_pipeline()` ran once on `df_excluded`, with incidents inside the target city's boundary removed via `geometry.union_all()` and `gdf_all.within()` [44] and once on unmodified `df_all`, recomputing both paths from scratch each time to isolate the local incident data effect.

3.2.3 Particle-Swarm Optimisation

The PSO implementation shared the Dijkstra and A* foundation (coordinate conversion, severity assignment, log-likelihood weighting and MultiDiGraph-to-DiGraph simplification) with a fixed NumPy seed of 42 for reproducibility [47]. Path quality was a weighted sum of normalised distance and risk, keeping the lambda weight interpretable [48], with Euclidean distance measuring node separation under a flat-surface assumption [49] and a greedy heuristic built from k-nearest-neighbour estimates directs the search toward likely goal-ward states, lowering computation [50]. Since PSO's continuous search risked premature convergence [51], genetic-algorithm operators countered it: mutation for exploration, swap for permutation testing and repair for reconnecting broken segments [52]. An A* fallback existed but never triggered. Outputs matched the other algorithms in terms of shortest and safest paths annotated with distance, travel time, severity risk and log-likelihood score, rendered as two colour-coded routes on the HTML map. The same before/after design applied: `run_routing_pipeline()` ran on `df_excluded` and `df_all`, recomputing both paths via `particle_swarm_path()` with `lambda_weight=1.0` and `0.0` respectively, reusing the concept-drift rationale, boundary-exclusion and command-line filtering above [42-44].

3.2.4 Ant-Colony Optimisation

The ACO implementation shared the PSO foundation (fixed seed, graph download, severity assignment and graph simplification) and like PSO scored path quality as a weighted sum of normalised distance and risk [48] with Euclidean node separation [49]. Pheromone values, the inverse-cost heuristic and a stagnation counter followed the canonical treatment of [53], the counter doubling as early stopping [53]. Ants traversed toward the goal node via nearest neighbours, scoring candidates as:

$$\text{Score} = (\tau^\alpha) (\eta + 0.5 \cdot \text{goal_bias})^\beta \quad (3)$$

where τ was the edge pheromone level, η was the inverse distance heuristic,

α and β controlled the pheromone vs heuristic influence `goal_bias` enabling the ACO to move towards the target with convergence on the best route resetting the counter. Pheromone dynamics combined evaporation (decaying each iteration to prevent unbounded accumulation), quality-weighted reinforcement on used edges and an elitist elite-boost on the best solution. An A* fallback guaranteed a feasible route where the colony failed to converge [53].

3.2.5 Wolf-Pack Optimisation

The WPO implementation shared the preceding algorithms' preprocessing (fixed seed, graph download, severity assignment and graph simplification) and initialised wolf paths by random-walk exploration on dynamic spatial networks [54]. Several operators balanced exploration and exploitation: loop-removal blocked revisiting a solution, a mutation operator forced deviation toward new paths and an alpha-guided crossover (`inherit_from_alpha`) drew candidates toward the leader wolf for stable convergence between risk and distance like adapting the genetic-operator strategy of an improved wolf-pack variant [55]. A local-refinement phase then searched each candidate's neighbourhood for lower-cost paths via NetworkX breadth-first shortest-path routines, giving distance- and risk-optimal traversal [56, 57]. The main loop followed the canonical scouting, calling and besieging procedure before returning the best path [55, 58], with an A* fallback should initialisation or the pathfinder fail. Routing outputs matched the preceding algorithms.

3.3 Results and analysis methodology

Negative log-likelihood (NLL) was chosen as the optimisation objective because it's the standard maximum-likelihood loss for probabilistic classifiers: minimising it rewards high probability on the correct class while penalising overconfident errors and as a strictly proper scoring rule it favours the well-calibrated estimates needed where reliable confidence and uncertainty quantification matter [59]. Precision, recall and F1 were excluded because in degenerate cases (no predicted or actual positives), each metric reduces to a division-by-zero form and therefore cannot be computed. Shortest-path accuracy was instead assessed against Google Maps.

$$\text{NLL} = -\sum \ln p(y/x) \quad (4)$$

Where p is the probability of encountering a high-risk edge and x is the path and y is the label indicating whether the path is optimal one based on the algorithm

3.4 Sanity Checks

The sanity tests confirmed the routing algorithms functioned correctly under typical operating conditions, matching real micromobility travel patterns in Ireland, where 70% of trips fall under 10 km [60]. Rather than unrealistic stress-test loads that would generate implausible distances and distorted risk outputs, the evaluation used two representative routes (one under 1 km, one above 10 km) confirming the system could execute core features in both the Command Line Interface and HTML map while still capturing risk appropriately [61].

4. Evaluation of the results and analysis

4.1 Results

Across 8 route pairs in 4 cities, all five algorithms were tested for shortest and safest paths under both non-real-time (single-dataset) and real-time (dual-case) configurations, with a repeatability check on the three stochastic methods to verify stability, variance behaviour and convergence consistency across independent runs.

Algorithm	Avg Safety Gain (%) (NRT/RT)	Avg Risk Reduction (%) (NRT/RT)	Avg Accuracy (%) (NRT/RT)	Avg Runtime (s) [Shortest/Safest] (NRT/RT)
Dijkstra	61.51/61.61	49.11/50	95.13/95.38	0.1/0.29 / 0.1/0.16
A*	11.29/10.77	-7.98/-7.09	94/96	0.1/0.01 / 0.11/0.01
PSO	61.86/44.32	50/35.44	95.38/95.13	0.19/0.18 / 0.13/0.15
ACO	63.43/58.04	50/56.91	94.63/95.63	3.4/4.41 / 0.16/ 0.11
WPO	61.98/62.3	50/50	95.75/95.75	10/6.4/ 7.58/ 9.04

Table 1: Overall results (Non Real-Time and Real-Time)



Figure 2: HTML map outcome (shortest (blue) and safest routes (cyan))

4.2 Analysis of the results

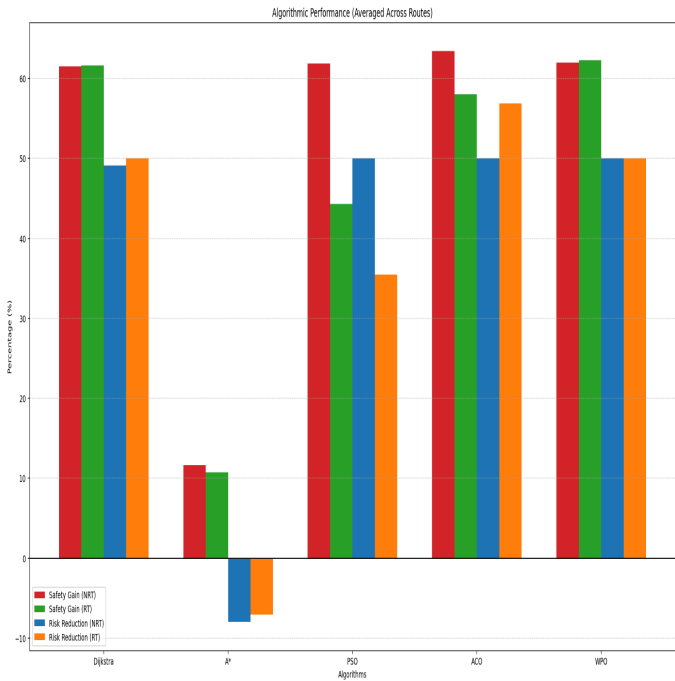


Figure 3: Merged bar chart showing each algorithm's safety improvement and risk reduction (Non-Real-Time and Real-Time)

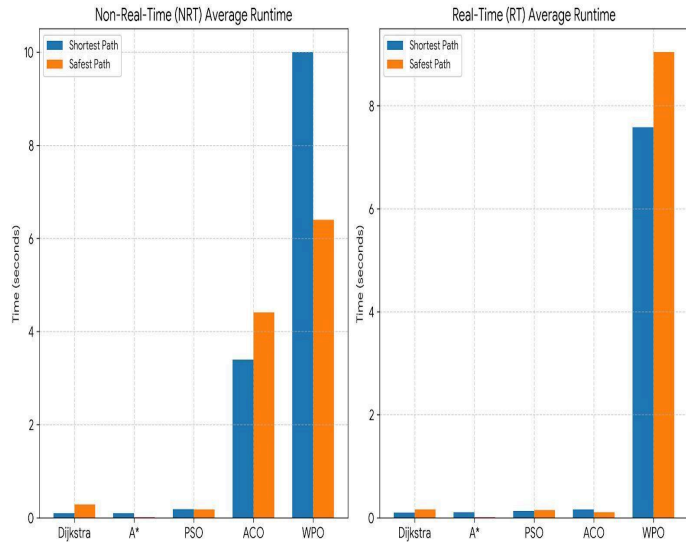


Figure 4: Merged bar chart showing each algorithm's path search average duration (Non-Real-Time and Real-Time)

Table 1 shows Dijkstra as the strongest static baseline, with the highest combined accuracy (~95%) and the lowest runtime, alongside a substantial average safety gain (~61%) and risk reduction (~49-50%) in both configurations. WPO achieved comparable or slightly

higher safety gains (~62%) and matched Dijkstra's risk reduction, but at the cost of the highest runtime by a wide margin (up to 10s for the shortest path alone), making it the most computationally expensive method despite strong safety outcomes.

ACO delivered the highest safety gain in the non-real-time setting (63.43%) and the highest risk reduction under real-time conditions (56.91%), but its runtime was highly inconsistent across configurations (3.4s shortest/0.16s safest, NRT vs. RT), reflecting the stochastic sensitivity of pheromone convergence discussed in Section 5.2.4.

PSO showed the clearest configuration-dependent shift: its safety gain dropped from 61.86% (non-real-time) to 44.32% (real-time), alongside a corresponding drop in risk reduction (50% to 35.44%), while its runtime remained the lowest among the three swarm methods in both settings. This combination of moderate safety performance and consistently low runtime made PSO the most balanced option once real-time constraints were introduced.

A* was the clear outlier: it was the only algorithm to return a negative average risk reduction in both configurations (-7.98% NRT, -7.09% RT), despite comparable or higher distance accuracy (94-96%) to the other methods. Its runtime was competitive with Dijkstra, confirming that A*'s underperformance wasn't a speed limitation but a systematic issue with how its heuristic interacted with the risk-weighted objective.

4.3 Discussions

A*'s negative risk reduction stems from its great-circle heuristic estimating distance to the goal regardless of whether the length or risk is being minimised. This biases the search toward paths that are heuristically "close" but not necessarily low-risk, unlike Dijkstra's unbiased, exhaustive expansion. The result is an algorithm that's fast but structurally unsuitable to risk-weighted objectives, answering part of RQ1: heuristic design, not just data availability, materially affects whether an algorithm can safely exploit incident data.

PSO's decline in both safety gain and risk reduction between non-real-time and real-time configurations, paired with its consistently low runtime, indicates that PSO trades some solution quality for speed once operating under the dual-case real-time pipeline - a reasonable trade-off for time-sensitive deployment, but one that shows performance isn't fully insulated from real-time constraints, addressing RQ1 directly.

ACO and WPO both achieved strong safety outcomes but through different costs: ACO's runtime volatility reflects its dependence on stochastic pheromone convergence, while WPO's high but more consistent runtime reflects its larger population and multi-stage refinement process. Neither is well suited to real-time deployment despite their safety benefits, whereas Dijkstra's strong static performance and PSO's real-time balance suggest a natural division of use cases: Dijkstra for pre-computed, static routing and PSO where routes must be recalculated frequently.

Taken together, these results answer RQ2: swarming algorithms (PSO, ACO, WPO) achieved safety gains broadly comparable to or exceeding Dijkstra's without requiring materially longer routes, confirming that safety improvement need not come at a large convenience cost. However, this benefit was not universal: A*'s consistent underperformance shows that algorithm choice, not swarm-based optimisation alone, determines whether this trade-off is favourable, reinforcing the need for algorithm-specific validation before deployment in real-world micromobility systems.

5. Conclusion and Future Work

The practicum showed that real-time data integration meaningfully influences routing performance and that swarming algorithms can enhance safety without imposing substantial convenience costs. Most routes achieved high accuracy and significant safety gains, often above 90% accuracy and up to 198% safety improvement, with remaining inaccuracies attributable to data availability constraints such as closed roads and limited OpenStreetMap coverage. Safety outcomes in the first test revealed minimal differences between Dijkstra and the swarming algorithms, while A* consistently underperformed due to its heuristic structure. Among the swarming methods, ACO offered modestly higher safety improvements but suffered from slow runtimes and weak local convergence. The second test confirmed that swarming algorithms deliver genuine safety benefits over classical methods. WPO achieved the highest safety improvements but at high computational cost: ACO again balanced safety with poorer distance optimisation: and PSO provided the most stable trade-off between shortest and safest routes despite the negative safety improvement between non-real-time and real-time. A* produced the weakest results, including routes that were less safe in their "safest" configuration. Both research questions were answered: RQ1 demonstrated that performance depended strongly on data availability and RQ2 showed that swarming algorithms can improve safety without materially compromising travel time or distance.

With more time, we could integrate the time of day in the shortest and safest route determination (alongside risk reduction, safety improvements and distance). We could look at incorporating a real-time API into the algorithms to show the route algorithms changing as situations arose, which would make our algorithms truly real-time. There could also be the use of accident data from the Road Safety Authority to enable more data to take the algorithms to the next level whilst also diversifying the algorithm testing in other locations in Ireland (not just the four largest cities). We would also look to integrate environmental and temporal factors (like weather or works) to really see the algorithmic adjustments.

So, to summarise, in this project these contributions to the subject area of micromobility safety analysis:

1. Taken data from the OpenStreetMap database and converted the coordinates into OSM IDs so that a map could be created
2. Implemented the conventional algorithms to establish a baseline for the swarming algorithms
3. Created the three swarming algorithms based on the data
4. Compared and contrasted algorithms based on computational efficiency (time length), safety improvements, distance accuracy and risk reduction
5. Used an unconventional outcome (negative log-likelihood) method to measure the safety improvements (instead of the usual F1 score or precision)

This represented a significant number of contributions to the field of micromobility system analysis and it now holds the potential for being applied to real-life micromobility systems as the usage continues to rise.

6. Student Contributions and Disclosures

The practicum was a joint effort with equal contribution from both members. BC sourced, cleaned and explored the accident dataset, extracting key variables such as outcome, coordinates and date. Using the standardised data, he implemented and tested the Dijkstra, A* and Wolf-Pack algorithms and authored the abstract, introduction, data limitations, algorithm descriptions, methodological sections and conclusion. HBB developed and tested the Ant-Colony and Particle-Swarm algorithms, conducted the analytical interpretation of results and wrote the background, algorithmic

parameters, results, analysis and discussion sections, as well as identifying appropriate supporting references.

The two Generative AI tools used in this final paper were Microsoft Copilot and Claude. This was mainly used in debugging the Python code, commenting the code at a higher standard and suggesting: ways to condense the paragraphs (this was done in Sections 2, 3, 4 and 5), appropriate references to back up certain points made (mainly in sections 3 and 4), ways to refine the phraseology of the paragraphs so that less space was used and so that the paragraphs would flow naturally from one to another (mainly in sections 4 and 5).

7. Appendix

Routes chosen to ensure that there was minimal bias (there must have been at least one severity-4 street in these routes)

Dublin 1: 34 North Great George's Street to Macartney Bridge, Baggot Street Lower

Dublin 2: Bram Stoker Park, Howth Road to Simmons Court Road (Elliott Gate 2)

Dublin 3: 37 Belgrave Road to James Joyce Park, James Joyce Street

Cork 1: Brookfield Mews, College Road to Waterstones, St. Patrick's Street

Cork 2: St. Patrick's Hill to Parliament Street

Limerick: Tait Place to Honan's Quay

Galway 1: Siobhan McKenna Road to Galway City Hostel, Frenchville Lane

Galway 2: 104 Father Griffin Road to Loyola Park, Lough Atalia Road

Practicum

Folder:

https://drive.google.com/drive/folders/1lpttGYr33S74NkpNavjIadoY2GyArHuV?usp=drive_link

8. References

- [1] T. Tamagusko *et al.*, "Data-Driven Approach for Urban Micromobility Enhancement through Safety Mapping and Intelligent Route Planning," *Smart cities*, vol. 6, no. 4, pp. 2035–2056, Aug. 2023
- [2] G. Yannis, *et al.*, "Safer Micromobility: Technical Background Report," 2024.
- [3] P. Crist, "Careful what you wish for... Practical implications of rules & requirements for shared micromobility," 2025.
- [4] M. Zakhem and J. Smith-Colin, "An E-scooter route assignment framework to improve user safety, comfort and compliance with city rules and regulations," *Transportation Research Part A: Policy & Practice*, vol. 179, pp. 103930–103930, 2024
- [5] S. Venu and M. Gurusamy, "A comprehensive review of path planning algorithms for autonomous navigation," *Results in Engineering*, vol. 28, pp. 107750–107750, Oct. 2025
- [6] L. Bayindir, "A review of swarm robotics tasks," *Neurocomputing*, vol. 172, pp. 292–321, Jan. 2016
- [7] Y. Liu and Y. Ouyang, "Planning charging stations for mixed docked and dockless operations of shared electric micromobility systems," *Transportation Research Part C Emerging Technologies*, vol. 171, pp. 104989–104989, Jan. 2025

- [8] N. Saum et al., "Optimizing Shared E-Scooter Operations under Demand Uncertainty: A Framework integrating Machine Learning and Optimization Techniques," *IEEE access*, pp. 1–1, Jan. 2024
- [9] S. Herr, "2024 State of the Industry Report," *North American Bikeshare & Scootershare Association*, Aug. 07, 2025.
- [10] J. Mahony, "Road Safety Authority National Survey on E-Scooters 2024," 2024.
- [11] "Micromobility: Data Challenges Associated with Assessing the Prevalence and Risk of Electric Scooter and Electric Bicycle Fatalities and Injuries," 2022.
- [12] S. Dikshit, et al, "The Use of Artificial Intelligence to Optimize the Routing of Vehicles and Reduce Traffic Congestion in Urban Areas," *EAI Endorsed Transactions on Energy Web*, vol. 10, no. 10.4108/ew.4613, Dec. 2023
- [13] M. Chau and K. Gkiotsalitis, "A systematic literature review on the use of metaheuristics for the optimisation of multimodal transportation," *Evolutionary Intelligence*, vol. 18, no. 2, Feb. 2025
- [14] Science Direct, "Swarm Intelligence Algorithm - an overview | ScienceDirect Topics,"
- [15] J. Kennedy and R. Eberhart, "Particle swarm optimization," *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, pp. 1942–1948, 1995
- [16] A. McKee, "Swarm Intelligence Algorithms: Three Python Implementations," *Datacamp.com*, Oct. 10, 2024.
- [17] T. Luo, et al, "An improved levy chaotic particle swarm optimization algorithm for energy efficient cluster routing scheme in industrial wireless sensor networks," *Expert Systems with Applications*, vol. 241, p. 122780, Nov. 2023
- [18] M. Dorigo et al., "Ant system: optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996
- [19] C. Liu et al., "AI-based 3D pipe automation layout with enhanced ant colony optimization algorithm," *Automation in Construction*, vol. 167, pp. 105689–105689, Nov. 2024
- [20] H.-S. Wu and F.-M. Zhang, "Wolf Pack Algorithm for Unconstrained Global Optimization," *Mathematical Problems in Engineering*, vol. 2014, pp. 1–17, 2014
- [21] H. Zhang, et al, "An Elite Wolf Pack Algorithm Based on the Probability Threshold for a Multi UAV Cooperative Reconnaissance Mission," *Drones*, vol. 8(9), p. 513, 2024
- [22] R. Mukha, et al, "Method and Software for Managing HR Processes of a Company using the Wolf Pack Algorithm," *2024 14th International Conference on Advanced Computer Information Technologies (ACIT)*, pp. 376–379, 2024
- [23] "Active Travel Collision Tracker," *Collisiontracker.ie*, 2018. (Accessed September 2025)
- [24] G. Niemeyer, "python-dateutil 2.9.0.post0 : Extensions to the standard Python datetime module," *PyPI*
- [25] R. Hamill, "A record €1 billion investment by Government since 2020 to prioritise safe walking and cycling for people," *Transport for Ireland*, Feb. 07, 2024.
- [26] G. Boeing, "Modeling and Analyzing Urban Networks and Amenities With OSMnx," *Geographical Analysis*, May 2025
- [27] E. Anello, "Geospatial Data Analysis with OSMnx | Towards Data Science," *Towards Data Science*, Jun. 16, 2023.
- [28] "User Reference - OSMnx 1.7.0 documentation," *osmnx.readthedocs.io*.
- [29] "pola-rs/polars," *GitHub*, Feb. 17, 2023.
- [30] python-visualization, "GitHub - python-visualization/folium: Python Data. Leaflet.js Maps.," *GitHub*, Jun. 16, 2025.
- [31] "Folium Documentation," *Readthedocs.io*, 2025.
- [32] P. E. Hart et al., "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968
- [33] G. A. Croes, "A method for solving traveling-salesman problems," *Operations Research*, vol. 6, no. 6, pp. 791–812, 1958
- [34] "Projections," *Openstreetmap.de*, 2026.
- [35] "MultiDiGraph—Directed graphs with self loops and parallel edges — NetworkX 3.6.1 documentation," *Networkx.org*, 2025.
- [36] S. Mukherjee, "Linear Probability Model: Definition, Examples & Limitations," *Digitalocean.com*, Sep. 17, 2025.
- [37] A. D. Tovar, "Negative log likelihood," *Deep Learning made easy*, Nov. 14, 2020.
- [38] M. Cryan, "Introduction to Algorithms and Data Structures Lecture 16: Dijkstra's Algorithm (for shortest paths) IADS -Lecture 16 -slide 1."
- [39] "dijkstra_path — NetworkX 3.2.1 documentation," *networkx.org*.
- [40] "Getting started — Folium 0.1.dev1+g9ab63f7 documentation," *python-visualization.github.io*.
- [41] J. Piribauer et al., "The variance-penalized stochastic shortest path problem," 2022.
- [42] H. D. Pham et al., "Reinforcement learning based estimation of shortest paths in dynamically changing transportation networks," *Frontiers in Future Transportation*, vol. 6, Mar. 2025
- [43] Lu, Jie, et al. "Learning Under Concept Drift: A Review." *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, 2018, pp. 1–1, 10.1109/tkde.2018.2876857.
- [44] "geopandas.GeoSeries.union_all — GeoPandas 1.1.2.dev107+g886c3f6bf.d20260626 documentation," *Geopandas.org*, 2026.
- [45] "User Reference - OSMnx 1.7.0 documentation," *osmnx.readthedocs.io*.
- [46] "astar_path — NetworkX 3.3 documentation," *networkx.org*.
- [47] S. A. Hassan et al., "Enhancing brain tumor detection through deep learning and explainable AI techniques," *Scientific Reports*, vol. 16, no. 1, p. 20558, July 2026, doi: 10.1038/s41598-026-60334-y.
- [48] G. Zhu et al., "Optimizing route planning via the weighted sum method and Multi-Criteria Decision-Making," *Mathematics*, vol. 13, no. 11, p. 1704, May 2025
- [49] Python Software Foundation, "Math — Mathematical Functions — Python 3.8.3rc1 Documentation," *docs.python.org*, 2024.
- [50] C. Wilt and W. Ruml. "Building a Heuristic for Greedy Search." *Proceedings of the International Symposium on Combinatorial Search*, vol. 6, no. 1, 1 Sept. 2021, pp. 131–140, 10.1609/socs.v6i1.18352.
- [51] A. Stacey et al., "Particle swarm optimization with mutation," The 2003 Congress on Evolutionary Computation, 2003. CEC '03., Canberra, ACT, Australia, 2003, pp. 1425-1430 Vol.2
- [52] C. Fernandes, "How to solve a Routing Problem with a Genetic Algorithm: A Practical Guide," *Data And Beyond*, Dec. 02, 2023.
- [53] M. Dorigo, "Ant colony optimization," *Scholarpedia*, vol. 2, no. 3, p. 1461, 2007
- [54] L. Régnier et al., "Universal exploration dynamics of random walks," *Nature Communications*, vol. 14, no. 1, Feb. 2023
- [55] X. Chen et al., "An improved Wolf pack algorithm for optimization problems: Design and evaluation," *PLoS ONE*, vol. 16, no. 8, p. e0254239, Aug. 2021, doi: 10.1371/journal.pone.0254239.
- [56] "CS 225 | BFS & DFS," *Illinois.edu*, 2026.
- [57] "shortest_path — NetworkX 3.4.2 documentation," *Networkx.org*, 2024.
- [58] W. Xu et al., "Wolf Pack Algorithm: An Overview," *Lecture notes in computer science*, pp. 96–110, Jan. 2025
- [59] E. D. Mitra and W. S. Hlavacek, "Parameter estimation and uncertainty quantification for systems biology models," *Current Opinion in Systems Biology*, vol. 18, pp. 9–18, Dec. 2019
- [60] "National Household Travel Survey 2022 Final Report," 2023.
- [61] "Comparing Functional vs Non-Functional Testing," *Virtuosqa.com*, 2025.